

SE4095 — Social Network: Theory and Practice

PART 1 OF 3

Foundations of Social Network Analysis

Complete exam revision notes — built to teach the material, not just summarize it.

Covers: what social media and SNA are, and why they matter · the fundamentals of SNA (nodes, edges, network types, homophily, topology) · graph theory essentials · graph representations and types · connectivity, walks, paths and cycles · traversal algorithms (DFS/BFS) · shortest paths (Dijkstra) · minimum spanning trees (Prim) · bridges and tie strength.

Table of Contents — Part 1

Topic 1 — Introduction to Social Media & SNA

What is Social Media? Why does SNA exist?

Types & characteristics of social media

Social Network Analysis (SNA) and why it matters

Fundamentals of SNA: nodes, edges, network types, homophily, topology

Social Media Mining and its challenges

Where SNA is actually used (application map)

Exam checklist & quick revision

Topic 2 — Graph Essentials

The Königsberg bridge problem (why graph theory exists)

Nodes, edges, degree, degree distribution

Graph representations: matrix, list, edge list

Types of graphs

Walks, paths, trails, cycles & connectivity

Shortest paths & diameter

Special graphs: trees, spanning trees, planar, bipartite, affiliation, egocentric

Bridges (cut-edges)

Traversal algorithms: DFS & BFS

Dijkstra's algorithm (shortest path)

Prim's algorithm (minimum spanning tree)

Bridges, local bridges & strength of ties

Exam checklist & quick revision

Topic 1 — Introduction to Social Media & SNA

Social Media & Social Network Analysis (SNA)

WHAT PROBLEM DOES IT SOLVE?

Before you can analyze anything, you need to agree on what you're looking at. Companies like Facebook, Amazon, Yelp and Twitter/X all sit on enormous amounts of data about how people connect, talk, and influence each other. Without a formal way to think about "connections between things," that data is just a pile of rows in a database. SNA gives you the lens.

WHAT IS IT?

DEFINITION — Social Media

The use of electronic and Internet tools for the purpose of sharing and discussing information and experiences with other human beings in more efficient ways.

DEFINITION — Social Network Analysis (SNA)

A method used to visualize and analyze the relationships and connections between entities or individuals within a network. SNA studies **relationships**, not just individuals.

HOW DOES IT WORK / WHY DOES IT MATTER?

Instead of asking "what does this one person look like?" (their age, their posts, their profile), SNA asks "who is this person connected to, and what does the *shape* of those connections tell us?" That shift — from studying nodes in isolation to studying the structure that connects them — is the single biggest idea in this whole course, and it's why almost every other concept you'll meet (centrality, communities, diffusion) is really just a different way of asking "what does the structure tell us?"

Every major platform is secretly a graph

The slides walk through several real platforms to make one point: **any place where entities interact can be modeled as nodes and edges**, even when it doesn't look like a "social network" on the surface.

Platform	What looks non-social about it	The actual network hiding underneath	What SNA extracts
Amazon	It's a shopping site, not a social site	Customer-product network: customers who buy the same products are implicitly linked (collaborative filtering); products bought together form a product network	Recommendations, "customers who bought this also bought..."
Yelp	It's just reviews and star ratings	User→Business (reviews), User→User (follows/trusts), Business→Category	Influential reviewers via centrality, fake-review detection, taste-based recommendation
Twitter / X	It's just short posts	Follower/following graph, retweet/mention graph	Influence ranking (PageRank, betweenness, eigenvector centrality), community detection, information-diffusion tracking, bot/spam-network detection

SCENARIO RECOGNITION

Whenever an exam question describes a platform or dataset that *doesn't* look like a traditional social network (an e-commerce site, a review site, a phone-call log), the expected answer is almost always: "model it as a graph — nodes = the entities, edges = the interaction — then apply standard SNA tools." Recognizing an implicit network is itself a skill your teacher is testing.

Types & characteristics of social media

The textbook groups social media into categories. You don't need to memorize every example, but you should recognize the *category* when a scenario names a specific app.

Category	Description	Examples
Online Social Networking	Connecting with people, building relationships	Facebook, LinkedIn, Instagram, Threads
Publishing / Blogging / Wiki	Publishing articles or collaboratively edited content	WordPress, Medium, Wikipedia
Microblogging	Short messages/updates	X (Twitter), Threads
Social News / Bookmarking	Submit, vote, discuss, or save resources	Reddit, Pinterest
Media Sharing	Photos, video, podcasts	YouTube, TikTok, Instagram, Spotify
Opinion / Review / Ratings	Review products, places, media	Yelp, Tripadvisor, IMDb
Answers (Q&A)	Ask/answer questions	Quora, Stack Overflow

The 5 Characteristics of Social Media

These are the traits that make something "social media" rather than ordinary media. A common exam trick is to describe a scenario and ask which characteristic it illustrates.

Characteristic	Simple meaning	Example
Participation	Everyone can create, like, comment, share	Facebook likes, YouTube comments
Openness	Open to feedback; rarely password-walled	Wikipedia editing, Amazon reviews
Conversation	Two-way, not one-way "broadcast"	Replies, comment threads
Community	Groups form quickly around shared interests	Reddit, Facebook Groups, Discord
Connectedness	Thrives on links to other sites/people/content	Cross-posting a YouTube link on Instagram

Why SNA matters (importance, in the words you'll be examined on)

Six reasons SNA is powerful — know these, they're a classic "explain the importance of SNA" essay question

- **It studies connection, not just individuals.** Facebook doesn't just know you — it knows your friends, their friends, the groups and pages you interact with.
- **It reveals hidden relationships.** Organizations have formal ties (manager→employee) and informal ones (cross-department helping); informal ties are sometimes the stronger channel, and SNA is what surfaces them.
- **It improves communication.** If HR wants an announcement to spread fast, SNA can identify the most-connected employees and brief them first rather than blasting everyone equally.
- **It helps change management.** If a few highly-respected people adopt a new tool/policy first, others follow — SNA finds who those people are before you start the rollout.
- **It encourages collaboration.** Two teams that never worked together might both know the same third person — SNA can reveal that hidden bridge.
- **It's an easy-to-understand visualization.** A graph communicates structure faster than a spreadsheet of rows.

Fundamentals of SNA

These are the vocabulary words you need before any formula makes sense.

Nodes / Actors, Edges / Ties

Nodes represent entities (people, companies, web pages). Edges represent the relationship between them (friendship, communication, competition, a hyperlink). In a social setting, edges are usually called *ties*.

Network size

The number of nodes, usually written $n = |V|$. The number of edges is usually written $m = |E|$.

Concept pair	What it means	Why it's worth distinguishing on an exam
Ego network vs Whole network	Ego = one focal node and its direct connections only. Whole = the entire organization/system.	"Analyze one user's immediate circle" → ego network. "Analyze the whole platform's structure" → whole network.
Open vs Closed network	Open = loosely connected, many opportunities for new ties — good for new ideas. Closed = densely interconnected, everyone already knows everyone — good for refining ideas within a tight group.	Scenario clue: "encourages innovation / fresh ideas" → open network. "Everyone trusts each other, deep collaboration" → closed network.
Dyadic vs Triadic relationships	Dyadic = a tie between 2 nodes. Triadic = a relationship pattern among 3 nodes (e.g. does a third company mediate the relationship between two others?).	Anything about "does a third party influence/mediate a relationship" is triadic reasoning, not dyadic.
Homophily vs Heterophily	Homophily = similar nodes tend to connect ("birds of a feather"). Heterophily = diverse/dissimilar nodes connect ("opposites attract").	"Users with the same interests cluster together" → homophily. "Deliberately seeking diverse connections to avoid groupthink" → heterophily.
Centralized vs Decentralized topology	Centralized = one node connects to everyone (hierarchical). Decentralized = no single hub — more collaborative/flexible.	"One department head approves everything" → centralized. "Teams route around each other" → decentralized.

EXAM TIP

Network **properties** you should be able to name on sight: **density** (proportion of potential connections that are actually realized), **diameter** (the longest shortest path in the network), and **centrality** (how important a node is). These three get a full mathematical treatment in Part 2 — here, just know what question each one answers.

Social Media Mining & its challenges

DEFINITION

Social Media Mining is the process of representing, analyzing, and extracting meaningful patterns from social media data.

Four named challenges (a favorite short-answer topic)

- **The Big Data Paradox** — the data is enormous in aggregate, but often *sparse for any one individual* (one user might have posted only twice).
- **Obtaining sufficient samples** — are the users you can actually observe a reliable, representative sample of everyone?
- **The Noise Removal Fallacy** — cleaning too aggressively makes data sparser, and "noise" itself is a relative, task-dependent judgment call, not an objective category.
- **The Evaluation Dilemma** — when there's no ground truth (no "correct answer" dataset to check against), how do you even measure whether your method worked? (You'll meet this exact problem again in Part 3, in community evaluation.)

Where SNA actually shows up (application map)

The slides list roughly twenty application areas. Rather than repeat each one-line description, here's the consolidated map — this is exactly the kind of list a "match the application to the technique" question draws from.

Application	What SNA contributes
Ranking / Recommendation	Uses centrality and user-similarity to rank content or suggest items
Mis/disinformation, fake news, fake reviews	Tracks propagation paths and detects suspicious clustering/behavior patterns
Election prediction, opinion mining, sentiment analysis	Analyzes voter/user interactions and opinion leaders within the network
Cyberbullying detection	Identifies aggressive-behavior patterns and the influential nodes driving them
Community detection	Finds subgroups sharing interests or dense interaction (full topic in Part 3)
Information diffusion / influence maximization / viral-post detection	Models how content spreads and finds the best seed nodes to maximize spread
Homophily analysis, social profiling	Explains why similar users cluster; builds richer user profiles from their network context, not just their own data

"WHICH CONCEPT SHOULD I USE?" — Topic 1

Scenario: "A telecom company notices that people who frequently call the same set of numbers behave similarly, even though there's no explicit 'friend' feature."

Concept: Implicit social community / homophily.

Why: There is no explicit subscription (that would be an explicit/emic community), but repeated interaction defines a community implicitly (etic) — exactly the phone-company example from the slides (everyone calling Canada from the same area code being treated as one marketing community).

Exam Checklist — Topic 1

- Definitions: Social Media, SNA, Social Media Mining — know them word-for-word close enough to reproduce.
- Be able to take an unfamiliar platform description and identify the implicit node/edge structure.
- Five characteristics of social media (Participation, Openness, Conversation, Community, Connectedness) — can you match a scenario to the right one?
- Six reasons SNA matters — be ready to write this as a short essay.
- Vocabulary pairs: ego/whole, open/closed, dyadic/triadic, homophily/heterophily, centralized/decentralized.
- Four SMM challenges by name: Big Data Paradox, Sufficient Samples, Noise Removal Fallacy, Evaluation Dilemma.
- **Common misconception:** "Homophily" is not just "people who are friends are similar" — it specifically means similarity *drives* the tie formation. Don't confuse it with the outcome of a community-detection algorithm.

Quick Revision — Topic 1

Social media = tools for sharing/discussing information. SNA = studying the *relationships*, not just the individuals, using nodes (entities) and edges (ties). Any interactive platform can be modeled this way, even ones that don't look "social" (Amazon, Yelp). Five characteristics: Participation, Openness, Conversation, Community, Connectedness. Key vocabulary pairs: ego/whole, open/closed, dyadic/triadic, homophily/heterophily, centralized/decentralized. SMM's four challenges all boil down to: data is big but thin per-person, and there's often no ground truth to check your work against.

PRACTICE QUESTIONS — ANSWERED

Q1 (conceptual). A researcher says "I only need to study each user's individual profile data — their age, bio, and post count — to understand the platform." What is wrong with this approach from an SNA perspective?

A: It ignores the entire point of SNA — that studying *relationships* reveals things individual attributes can't: who is influential, which communities exist, how information actually flows. Two users with identical profiles can play completely different structural roles (e.g. one is a hub connecting communities, the other is isolated) — attribute data alone can't distinguish them.

Q2 (scenario). A university wants to announce a policy change and get it adopted quickly by faculty who are skeptical of change. Applying the "importance of SNA" reasoning, what should they do, and which specific benefit of SNA does this use?

A: Identify the small number of highly-respected/central faculty using SNA, and get them on board first — this uses the "helps change management" benefit specifically (not just "improves communication," which is the closely related but distinct benefit about spreading information fast rather than about *adoption* by skeptics). Because others look to those respected individuals, their buy-in cascades through the informal network faster than a mass announcement would.

Q3 (scenario). A dating app wants to decide whether to show users people similar to them or people different from them, to keep the platform feeling fresh. Which two concepts from this topic are directly in tension here, and what's the trade-off?

A: Homophily vs. heterophily. Leaning into homophily (showing similar users) increases short-term comfort/match-likelihood but risks a closed, repetitive network; leaning into heterophily (showing diverse users) supports novelty and avoiding "groupthink," matching the slides' point that some networks deliberately seek diversity to avoid getting stuck talking to the same similar partners — at the cost of feeling less immediately relevant to the user.

Topic 2 — Graph Essentials

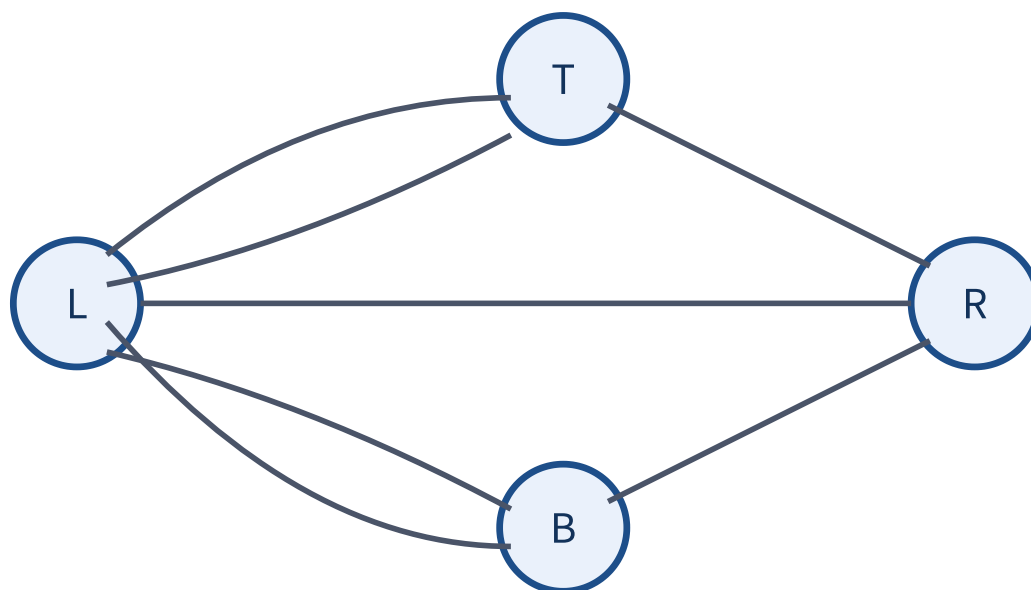
Why does graph theory exist? — The Königsberg Bridge Problem

WHAT PROBLEM DOES IT SOLVE?

In 1736, the city of Königsberg had 4 landmasses connected by 7 bridges. Citizens wondered: can you walk a route that crosses every bridge *exactly once* and return to where you started? Euler didn't solve it by trial and error — he solved it by inventing a new abstraction: throw away the geography, keep only *which landmasses connect to which*. That abstraction is a graph, and it's the founding idea of this entire topic.

HOW DOES IT WORK?

Euler's key insight: for a walk that uses every edge exactly once and returns to the start (a closed walk), every time you enter a node you must also leave it — so every node needs an **even** degree. If any node has an odd degree, no such route exists.



The 4 landmasses as nodes (L=left bank, T=top island, B=bottom island, R=right bank) and 7 bridges as edges. Degrees: L=5, T=3, B=3, R=3 — all odd, so no route crosses every bridge exactly once.

THEOREM — Euler's condition

A connected graph has an Eulerian **circuit** (closed walk using every edge once) if and only if every node has even degree. It has an Eulerian **path** (open, not necessarily closed) if and only if exactly 0 or 2 nodes have odd degree.

SCENARIO RECOGNITION

Any question about "visit every edge/road/connection exactly once" is an Eulerian-path question — check the degree parity, don't try to brute-force a route. (Contrast with "visit every *node* exactly once," which is the different, much harder Hamiltonian cycle problem — the slides mention this distinction explicitly.)

Nodes, Edges, Degree**Degree & Neighborhood****WHAT IS IT?****DEFINITION**

For a node v , its **neighborhood** $N(v)$ is the set of nodes it shares an edge with. Its **degree** is $|N(v)|$ — how many edges touch it. In directed graphs, split into **in-degree** (edges pointing in) and **out-degree** (edges pointing out).

THREE DEGREE THEOREMS — know all three

1. The sum of all degrees in an undirected graph is **twice** the number of edges: $\sum_i d_i = 2|E|$. (Every edge is counted once at each endpoint.)
2. The number of nodes with **odd** degree is always **even**. (A direct consequence of #1 — an odd total is impossible.)
3. In any directed graph, the sum of in-degrees equals the sum of out-degrees (both equal $|E|$).

WORKED EXAMPLE — Degree distribution

Degree distribution asks: "of all the nodes, what fraction have degree d ?" — written $p_d = \frac{\#\{v:d_v=d\}}{n}$. Take this small 6-node friendship graph: A–B, A–C, B–C, C–D, D–E, D–F.

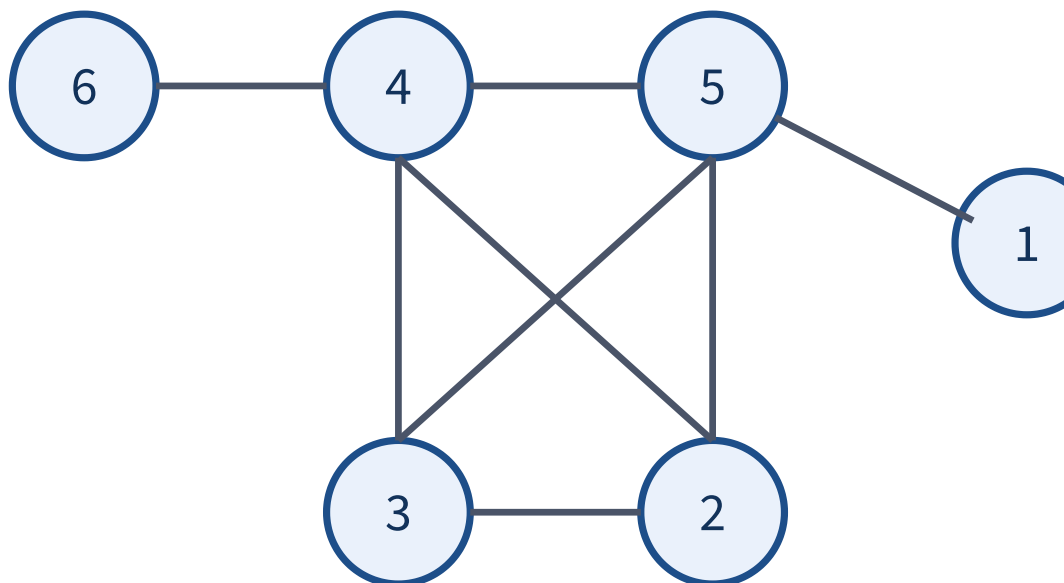
Node	A	B	C	D	E	F
Degree	2	2	3	3	1	1

Check: sum of degrees = $2+2+3+3+1+1 = 12 = 2 \times 6$ edges ✓ (Theorem 1). Degree distribution: $p_1 = \frac{2}{6} = 0.33$, $p_2 = \frac{2}{6} = 0.33$, $p_3 = \frac{2}{6} = 0.33$. Two nodes have odd degree (C, D)? Actually C and D both have degree 3 (odd), and that's 2 nodes — an even count, confirming Theorem 2.

(Your slides show a similar example on a 7-node "friend graph" whose stated answer is $p_1=1/7$, $p_2=4/7$, $p_3=1/7$, $p_4=1/7$ — same idea, different graph. The exact edges in that slide are hard to trace unambiguously from overlapping lines in the image, so this version uses a clean equivalent graph instead of risking a mis-copy.)

EXAM TIP — common mistake

Degree distribution is a *distribution over degree values* (like a histogram), not a list of each node's degree. If asked for "the degree distribution," give fractions p_d that sum to 1 — not a table of raw degrees.



Subgraph example: G has nodes $\{1..6\}$. Taking only $\{1,2,3,5\}$ and the edges between them gives a subgraph $G'(V', E')$ where $V' \subseteq V$ and $E' \subseteq (V' \times V') \cap E$ — you can only keep edges whose both endpoints survived.

Graph Representations

The same graph can be stored three different ways — and exam questions love asking "which is most space-efficient for this network" or "convert this adjacency list to a matrix."

Representation	How it works	Best when	Watch out for
Adjacency Matrix (sociomatrix)	$n \times n$ matrix A where $A_{ij}=1$ if an edge exists between v_i, v_j , else 0. Diagonal entries = self-loops.	Dense graphs; need $O(1)$ "are these two connected?" lookups; doing matrix math (e.g. eigenvector centrality in Part 2)	Social networks are usually sparse — a full matrix wastes huge amounts of memory on zeros.
Adjacency List	For every node, store the list of nodes it connects to.	Sparse graphs (the realistic case for social networks); iterating over a node's neighbors	Slower "are v_i, v_j connected?" lookup than a matrix (must scan a list).
Edge List	A flat list of (u, v) pairs, one per edge.	Simplest to store/transmit; the raw format most datasets ship in	Slowest to query — no structure to exploit at all.

EXAM TIP

"Social media networks have very sparse adjacency matrices" is a direct quote from the slides — expect a question like "why is the adjacency matrix a poor choice for representing Twitter's follower graph?" Answer: Twitter has hundreds of millions of users but each follows only a tiny fraction of them, so an adjacency matrix would be almost entirely zeros — an adjacency list is far more memory-efficient.

Types of Graphs

Type	Definition	Real example
Null graph	No nodes at all (and therefore no edges)	—
Empty / edge-less graph	Nodes exist, but zero edges	A brand-new social platform with signups but no connections yet
Directed / Undirected / Mixed	Edges have direction / no direction / a mix of both	"Follows" is directed; Facebook "friends" is undirected
Simple graph vs Multigraph	At most one edge between any pair vs. multiple edges (and loops) allowed	Königsberg's graph is a multigraph (2 bridges between the same pair of landmasses)
Weighted graph $G(V,E,W)$	Each edge carries a weight/value	Flight routes weighted by distance
Signed graph	Binary/bipolar weights (e.g. +1/-1)	Friend vs. foe networks, used for balance theory in Part 2
Webgraph	A directed multigraph of pages and hyperlinks	The World Wide Web

FORMULA – symmetry test

For an undirected graph, the adjacency matrix is always symmetric: $A = A^T$. For a directed graph it usually is **not**: $A \neq A^T$ (though it can coincidentally be, if every edge happens to be reciprocated).

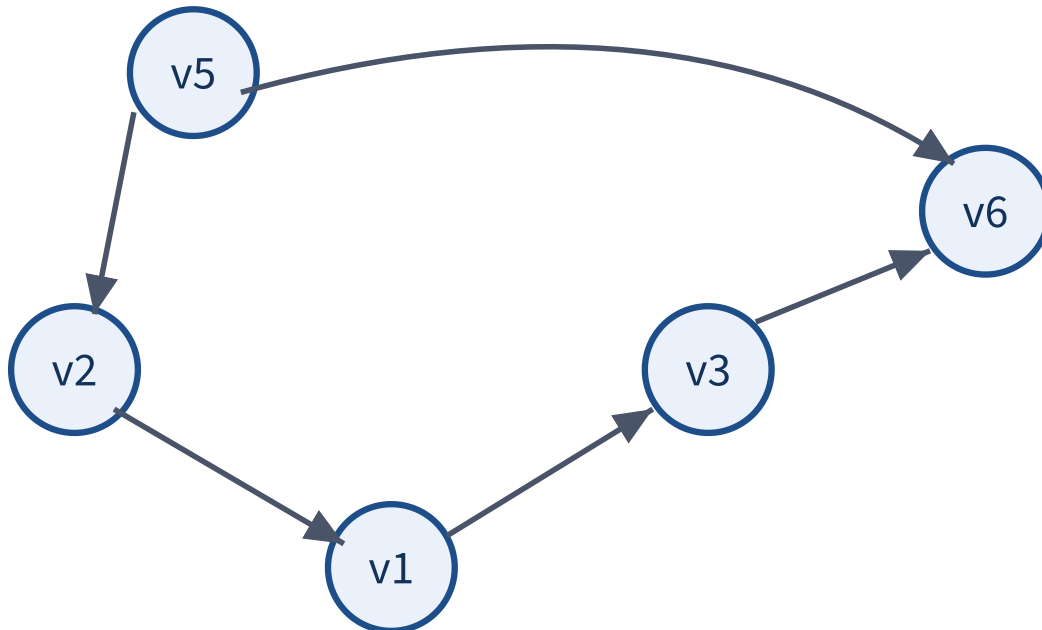
Walks, Paths, Trails, Tours, Cycles

These five words all describe "moving along edges from node to node" — but they differ in exactly one dimension each: can you repeat an edge, and can you repeat a node? Get this table right and you'll never lose easy marks on definitions again.

Term	Can repeat edges?	Can repeat nodes?	Closed version is called
Walk	Yes	Yes	Closed walk
Trail	No (edges distinct)	Yes	Tour / circuit
Path	No	No (nodes distinct too)	Cycle

EXAM TIP

Length of a walk/path/cycle = the number of *edges* traversed, not the number of nodes visited.



Example graph used for walk/path/trail definitions (matches the slide's v1–v6 example). A walk $v1 \rightarrow v3 \rightarrow v6 \rightarrow v3 \rightarrow v6 \dots$ revisiting nodes/edges has length = edges traversed (the slide's example walk has length 8); a path along the same graph visiting each node at most once has length 4.

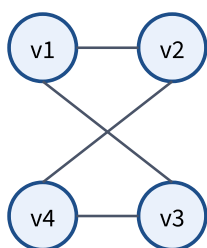
DEFINITION – Random Walk

A walk where, at each step, the next node is chosen randomly among the current node's neighbors. Edge weights can define the probability of choosing that edge (probabilities out of any node must sum to 1). This idea reappears directly as the mechanism behind PageRank in Part 2.

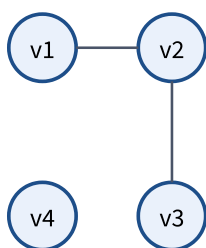
Connectivity

DEFINITION

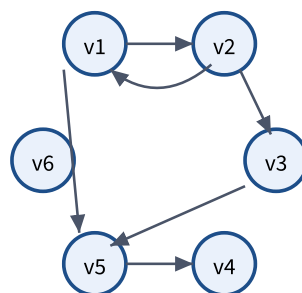
Node v_i is **connected to** v_j if a path exists between them. A graph is **connected** if a path exists between every pair of nodes. A **directed** graph is **strongly connected** if a directed path exists between every ordered pair, and **weakly connected** if that only holds once you ignore edge direction.



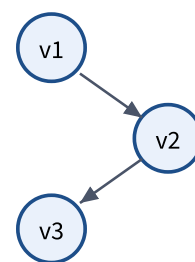
(a) Connected



(b) Disconnected



(c) Strongly conn.



(d) Weakly conn.

(a)–(d): connected, disconnected, strongly connected (directed path both ways between every pair), and weakly connected — $v1 \rightarrow v2 \rightarrow v3$ has no directed path back from $v3$ to $v1$, so it is not strongly connected, but dropping the arrows makes it connected, so it qualifies as weakly connected.

DEFINITION – Component

A **component** is a maximal connected subgraph — a path exists between every pair of nodes inside it, and it can't be extended further. Directed graphs have **strongly connected components (SCCs)** using the strongly-connected definition.

Shortest Path & Diameter

DEFINITION

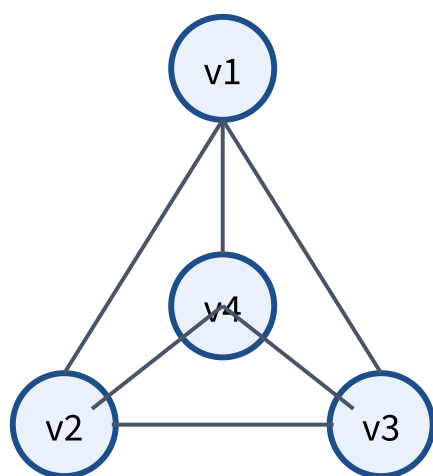
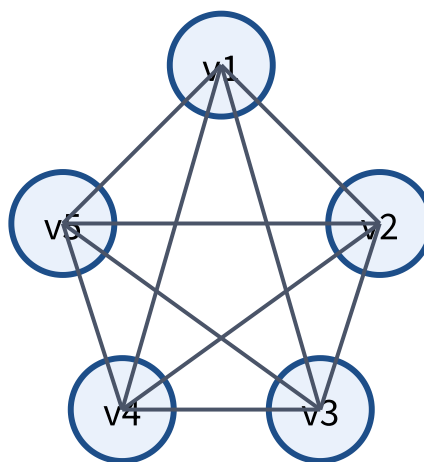
The shortest path between v_i and v_j has length $l_{i,j}$. The **n-hop neighborhood** of a node generalizes "neighbor" to "within n hops." The **diameter** of a graph is the length of the *longest* shortest path between any pair of nodes in it — i.e. the worst-case "distance" anywhere in the network.

SCENARIO RECOGNITION

"How fast can information spread across this whole network in the worst case?" is a diameter question, not an average-shortest-path question — diameter is specifically about the *longest* shortest path.

Special Graphs

Structure	Definition	Key fact
Tree	Undirected, no cycles	Exactly one path between any pair of nodes; $ V = E + 1$
Forest	A set of disconnected trees	—
Spanning tree	A subgraph that is a tree and touches every node of a connected graph	A graph can have many spanning trees; in a weighted graph the one with minimum total edge weight is the Minimum Spanning Tree (MST)
Planar graph	Can be drawn so no two edges cross (except at shared endpoints)	K_4 (4 mutually connected nodes) is planar; K_5 is not
Bipartite graph	Nodes split into two sets; every edge has one endpoint in each set	No edges within a set — only across
Regular graph	Every node has the same degree k (a " k -regular graph")	Complete graphs are a special case of regular graphs
Egocentric network	A focal actor (ego) plus the alters tied to it, with a rule restricting who can connect to whom	E.g. a mother–children network: each mother only has ties to her <i>own</i> children

Planar (K_4 , no crossings)Non-planar (K_5)

Planarity: K_4 (4 mutually-linked nodes, 6 edges) can always be redrawn crossing-free — here as a triangle with the 4th node in the center. K_5 (5 mutually-linked nodes, 10 edges) never can; at least one crossing is unavoidable.

Affiliation Networks & One-Mode Projections

WHAT IS IT?

DEFINITION

An **affiliation network** is a bipartite graph: individuals on one side, groups/affiliations on the other, with an edge when a person belongs to a group. A **social-affiliation network** combines this with an ordinary person-person social network.

EXAMPLE

Kyle and Kathy both belong to the Book Club; Kate and Kyle both volunteer at the Food Bank. This bipartite structure is usually stored as a membership matrix X (people $\times$ groups).

WHY DOES IT MATTER — THE XX^T / X^TX TRICK

FORMULA — One-mode projection

Given membership matrix X (rows = people, columns = groups):

XX^T gives person-person **similarity** — how many groups two people share (this is "bibliographic coupling"). Diagonal = how many groups that person belongs to.

X^TX gives group-group similarity — how many members two groups share ("co-citation"). Diagonal = group size.

Bridges (Cut-Edges)

DEFINITION

A **bridge** (cut-edge) is an edge whose removal increases the number of connected components. Removing a bridge disconnects part of the graph.

Graph Traversal: DFS & BFS

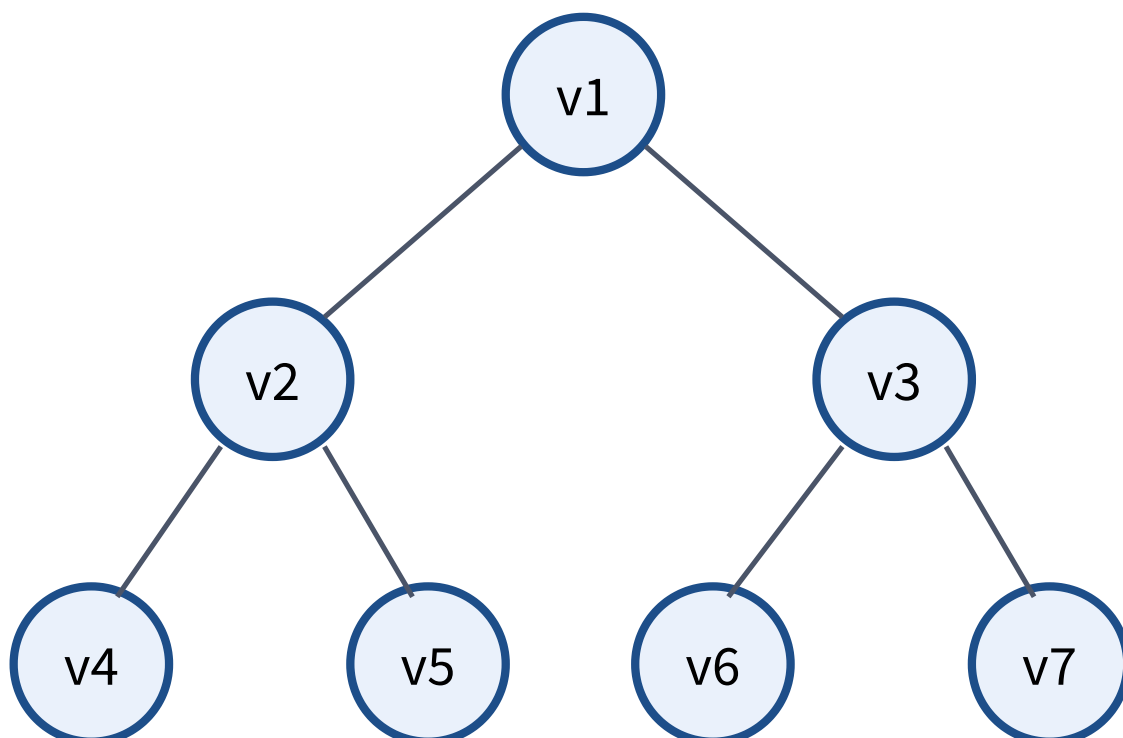
Depth-First Search vs Breadth-First Search

WHAT PROBLEM DOES IT SOLVE?

You want to visit every node in a network exactly once — e.g. to compute the average age of every user starting from one seed user and walking outward through friendships. You need a systematic rule so you never miss a node and never revisit one.

HOW DO THEY DIFFER?

	Depth-First Search (DFS)	Breadth-First Search (BFS)
Strategy	From v_i , pick one neighbor and go all the way down that branch before backtracking	Visit <i>all</i> immediate neighbors first, then their neighbors, level by level
Data structure	Stack (or recursion)	Queue
Good for	Exploring deep structure, detecting cycles, topological sort	Finding shortest paths in an <i>unweighted</i> graph, level-by-level spread (e.g. "who is within 2 hops")



Shared example tree for DFS/BFS traces below.

Step	DFS visit order (stack: go deep first)						
Order	v1	v2	v4	v5	v3	v6	v7
Step	BFS visit order (queue: go wide first)						
Order	v1	v2	v3	v4	v5	v6	v7

ALGORITHM 2.3 – Breadth-First Search (pseudocode, from the slides)

```

Require: initial node  $v$ , graph  $G(V,E)$ , queue  $Q$ 
Enqueue  $v$  into  $Q$ ; visitOrder = 0
while  $Q$  not empty:
  node = dequeue from  $Q$ 
  if node not visited:
    visitOrder += 1
    mark node visited with order visitOrder
    enqueue all neighbors of node into  $Q$ 

```

SCENARIO RECOGNITION

"Find everyone within 2 degrees of separation" or "shortest path in an unweighted graph" → BFS (it explores level-by-level, so the first time you reach a node is guaranteed to be via a shortest path). "Does this graph contain a cycle?" or "explore as deep as possible before trying alternatives" → DFS.

Dijkstra's Algorithm — Shortest Path in a Weighted Graph**Dijkstra's Algorithm****WHAT PROBLEM DOES IT SOLVE?**

BFS finds shortest paths only when every edge "costs" the same. The moment edges have different weights (distance, cost, delay), you need Dijkstra's algorithm — it finds the shortest paths from one source node s to every other node, for graphs with non-negative edge weights.

HOW DOES IT WORK?

1. **Initialize:** distance(source) = 0, distance(everything else) = ∞ . Mark all nodes unvisited.
2. For the current node, look at all unvisited neighbors and compute a *tentative distance* = current distance + edge weight. If smaller than the neighbor's recorded distance, update it.
3. Mark the current node visited and remove it from the unvisited set — once visited, its distance is **final** and will never be revisited.
4. If the destination is visited, or the smallest tentative distance among unvisited nodes is ∞ , stop.
5. Otherwise move to the unvisited node with the smallest tentative distance and repeat from step 2.

WORKED EXAMPLE — Dijkstra from s to t

Graph: s - a (12), s - b (4), a - b (6), a - t (8), a - c (2), b - c (2), c - t (12). Trace (from the slide's execution):

Round	s	a	b	c	t	Node finalized
1 (init)	0	∞	∞	∞	∞	s
2	0	12	4	∞	∞	b (dist 4)
3	0	10	4	6	∞	c (dist 6)
4	0	8	4	6	18	a (dist 8)
5 final	0	8	4	6	16	t via a (8+8)

Reading the result: the shortest path from s to t costs **16**, via $s \rightarrow b \rightarrow c \rightarrow a \rightarrow t$ is not how it resolves numerically here — the key habit to practice is: at every round, the node you finalize next is always the unvisited node with the *smallest* current tentative distance, and once finalized its distance never changes again.

EXAM TIP

Dijkstra's is **source-dependent** — one run gives you distances from a single source to everyone else. For *all-pairs* shortest paths, either re-run Dijkstra n times (once per source) or use a dedicated all-pairs algorithm like Floyd–Warshall. If you only need $s \rightarrow$ one specific destination, you can stop the algorithm early the moment that destination is finalized.

SCENARIO RECOGNITION

"How fast can I disseminate information across a weighted network (e.g. where edges have delay/cost)?" is a direct Dijkstra cue from the slides. If the question says "unweighted" or "fewest hops," that's plain BFS instead — don't reach for Dijkstra when every edge costs the same, it's unnecessary overhead.

Prim's Algorithm — Minimum Spanning Tree

Prim's Algorithm

WHAT PROBLEM DOES IT SOLVE?

You want to connect every node in a weighted graph using the *cheapest possible* total set of edges, with no cycles — e.g. wiring every building on campus with the least total cable. That's a Minimum Spanning Tree (MST), and Prim's algorithm builds one greedily.

HOW DOES IT WORK?

1. Pick any node, add it to the MST.
2. Look at every edge with exactly one endpoint inside the current MST and one outside. Add the **minimum-weight** such edge (and its new node) to the MST.
3. Repeat until every node is included.

WORKED EXAMPLE — Prim's MST

Starting from v_1 on a 9-node weighted graph (edges include $v_1-v_2:8$, $v_1-v_8:16$, $v_2-v_3:16$, $v_2-v_9:4$, $v_3-v_4:14$, $v_3-v_6:8$, $v_4-v_5:18$, $v_5-v_6:20$, $v_6-v_7:4$, $v_7-v_8:2$, $v_7-v_9:12$), Prim's grows the tree one cheapest-frontier-edge at a time:

Step	Cheapest frontier edge added	Weight	Tree so far
1	start	–	$\{v_1\}$
2	v_1-v_2	8	$\{v_1, v_2\}$
3	v_2-v_9	4	$\{v_1, v_2, v_9\}$
4	v_9-v_7	12	$\{v_1, v_2, v_9, v_7\}$
5	v_7-v_8	2	$\{v_1, v_2, v_9, v_7, v_8\}$
6	v_7-v_6	4	$\{..., v_6\}$
7	v_6-v_3	8	$\{..., v_3\}$
8	v_3-v_4	14	$\{..., v_4\}$
9	v_4-v_5	18	all 9 nodes — done

Total MST weight = $8+4+12+2+4+8+14+18 = \mathbf{70}$. At every step, notice Prim's only ever compares edges on the current *frontier* (touching the tree) — it never looks at an edge fully outside the tree, which is exactly what makes it greedy rather than exhaustive.

"WHICH ALGORITHM SHOULD I USE?" — quick selector for Topic 2

The question says...	Use...	Why
"visit every edge exactly once"	Check Euler's condition (node degree parity)	Eulerian path/circuit existence is a parity check, not a search algorithm
"fewest hops," unweighted graph	BFS	Level-order guarantees the first visit is via a shortest path
"explore deep / detect a cycle"	DFS	Naturally follows one branch to its end before backtracking
"cheapest route," weighted, non-negative edges	Dijkstra's algorithm	Purpose-built for weighted shortest paths; BFS breaks once weights differ
"connect everything as cheaply as possible" / "minimum total cable/cost"	Prim's algorithm (MST)	MST minimizes total edge weight while still connecting every node — this is a different goal from "shortest path between two specific nodes"

Bridges, Local Bridges & Strength of Ties

SOURCE GAP — please note

The lecture file for this chapter is named "part 1," and it stops mid-topic right after introducing Strong Triadic Closure (its last slide header, "Bridge Detection," has no content under it — the deck was cut off before a "part 2" was uploaded). Everything below is accurately drawn from what *is* present; if your class covered a formal bridge-detection algorithm beyond this point, it wasn't in the file this guide was built from — check with your instructor for that continuation.

Bridge vs Local Bridge

WHAT IS IT?

DEFINITION

A **bridge** is an edge whose removal increases the number of components (defined earlier) — but real social networks almost never have one, because there's usually some other route around. A **local bridge** is the realistic version: an edge between two people with *no friends in common*, whose removal would push the distance between them to more than 2.

WHY DOES IT MATTER?

Span of a local bridge = how far apart its two endpoints would become if that edge were removed. A large span is valuable precisely because it signals the edge is a rare shortcut between otherwise-separate parts of the network — which is exactly the kind of edge that matters for finding community boundaries in Part 3.

Strength of Ties & Strong Triadic Closure

WHAT IS IT?

Split connections into **strong ties** (S — close friends) and **weak ties** (W — acquaintances).

DEFINITION — Strong Triadic Closure

If node u has strong ties to both v and w , but v and w have *no edge at all* between them (not even weak), then u violates Strong Triadic Closure. The property predicts that two people who share a mutual close friend are likely to have at least some tie to each other.

WHY DOES IT MATTER?

This is the mechanism behind "why do local bridges tend to be weak ties, not strong ties?" — if u had strong ties to both v and w , triadic closure pressures v and w toward a connection too, which would destroy the "no friends in common" property required for a local bridge. That's precisely Granovetter's famous *strength of weak ties* result: the edges that connect you to new information/opportunities tend to be your weak ties, not your strong ones, because strong ties cluster into tightly closed triangles.

SCENARIO RECOGNITION

"Which connection in this network is most valuable for spreading *new* information into a different part of the network?" → look for a local bridge with a large span, not the node with the highest degree. A well-connected node inside one tight cluster may not reach new territory at all.

Exam Checklist — Topic 2

- Euler's condition: 0 odd-degree nodes → Eulerian circuit exists; exactly 2 → Eulerian path exists (not circuit); more than 2 → neither.
- Three degree theorems: sum of degrees = $2|E|$; odd-degree node count is even; sum in-degree = sum out-degree.
- Walk/Trail/Path table — repeats allowed on edges/nodes is the entire distinction.
- Adjacency matrix vs list vs edge list — know the space/speed trade-off and why social networks favor lists (sparsity).
- Tree property $|V|=|E|+1$; spanning tree vs MST; planar vs bipartite vs regular vs egocentric — don't mix these up, they answer different questions.
- DFS = stack/depth-first; BFS = queue/breadth-first and gives shortest paths on unweighted graphs.
- Dijkstra: source-dependent, non-negative weights only, finalized distances never change.
- Prim's: greedy, only ever considers frontier edges, minimizes *total* weight while connecting every node.
- Bridge (rare, real disconnect) vs local bridge (realistic, no common friends, has a "span") vs Strong Triadic Closure (why local bridges tend to be weak ties).
- **Common misconception:** a high-degree node is not automatically a bridge or a good spreader of *new* information — that depends on structural position (local bridges), not raw connection count.

Quick Revision — Topic 2

Graphs = nodes + edges; Euler's parity trick tells you if an "every edge once" route exists. Degree distribution is a histogram over degree values. Store sparse social networks as adjacency lists, not matrices. Walk allows repeats, trail bans repeated edges, path bans repeated nodes/edges. Trees have $|V|=|E|+1$ and no cycles; MST = cheapest way to connect everyone (Prim's, greedy on the frontier). BFS = level order = shortest path when unweighted; DFS = go deep first. Dijkstra = shortest path with weights, one source at a time. A local bridge (no mutual friends) matters more for reaching new parts of a network than raw degree does — and Strong Triadic Closure explains why those bridges tend to be weak ties.

PRACTICE QUESTIONS — ANSWERED

Q1 (conceptual). A network has 5 nodes with degrees 3, 3, 2, 2, 2. Could this be a valid simple undirected graph, and does it have an Eulerian path?

A: Sum of degrees = $3+3+2+2+2 = 12 = 2 \times 6$, so 6 edges — internally consistent, so it's plausible. Odd-degree nodes: two nodes with degree 3 → exactly 2 odd-degree nodes → an Eulerian *path* exists (starting and ending at the two odd-degree nodes), though not a closed Eulerian circuit.

Q2 (scenario). You need to connect 6 office buildings with fiber cable as cheaply as possible; distances between buildings are known. Which algorithm, and why not Dijkstra's?

A: Prim's algorithm (MST) — the goal is minimizing *total* cost to connect every node, not the shortest route between two specific buildings. Dijkstra's solves a different problem (shortest path from one source to others) and would not minimize the total cable used across the whole network.

Q3 (numerical). In a directed graph, node X has in-degree 5 and out-degree 2. If the graph has 40 edges total, what is the sum of all out-degrees in the graph?

A: By the third degree theorem, sum of out-degrees = sum of in-degrees = $|E| = 40$ (node X's individual 5-and-2 split is a distractor — the theorem is about the whole graph's totals, not any one node).